

Challenge 1 – Student Record Application

Problem Statement

A college application stores student records that may not be adequately protected.

1. Identify Confidentiality Vulnerabilities

1. Student records are stored without encryption, allowing anyone with database access to read names, marks, addresses and phone numbers.
2. Weak authentication such as default passwords or short passwords allows attackers to guess credentials.
3. Improper input validation can lead to SQL Injection, exposing the complete database.

2. Explain the Attack Scenario

Step 1: An attacker visits the login page and enters SQL injection payloads such as ' OR '1'=1.

Step 2: If successful, the application bypasses authentication and grants access to student records.

Step 3: If database files are copied from the server, all information is readable because it is stored in plaintext.

3. Demonstration (Using Only the Provided Test Environment)

Within the provided test environment, attempt SQL Injection only on the test login page. Observe whether authentication is bypassed and whether student records become visible. Do not perform this on any real system.

4. Recommended Secure Countermeasures

- Encrypt sensitive records using AES-256.
- Store passwords using encrypt or Argon2 hashing.
- Use parameterized queries/prepared statements.
- Validate and sanitize all user inputs.
- Implement Role-Based Access Control (RBAC).

Conclusion

Student information is confidential. Strong encryption, secure coding practices and proper authentication significantly reduce the risk of unauthorized disclosure.

Challenge 2 – Confidential Notes Application

Problem Statement

Users believe unauthorized people can read their confidential notes.

1. Identify Confidentiality Vulnerabilities

1. Notes are stored in plaintext.
2. The application fails to verify whether the logged-in user owns a note (Broken Access Control/IDOR).
3. Weak session management may allow session hijacking.

2. Explain the Attack Scenario

Step 1: An attacker changes the note ID in the URL to access another user's notes.

Step 2: If session cookies are stolen, the attacker can impersonate the victim.

Step 3: Direct database access reveals all notes if they are unencrypted.

3. Demonstration (Using Only the Provided Test Environment)

In the test environment, modify a sample URL such as /notes?id=1 to /notes?id=2. If another user's note appears, the application has an authorization flaw.

4. Recommended Secure Countermeasures

- Encrypt all stored notes.
- Verify ownership before displaying any note.
- Use secure session cookies and HTTPS.
- Implement RBAC.
- Automatically expire inactive sessions.

Conclusion

Proper authorization and encryption ensure that only the owner can access confidential notes.

Challenge 3 – Hospital Patient Information

Problem Statement

A hospital application stores patient records believed to be secure.

1. Identify Confidentiality Vulnerabilities

1. Medical records stored without encryption.
2. Weak administrator authentication.
3. SQL Injection vulnerability.

2. Explain the Attack Scenario

Step 1: An attacker exploits SQL Injection to retrieve patient details.

Step 2: Weak administrator passwords permit unauthorized login.

Step 3: Unencrypted backups expose patient data.

3. Demonstration (Using Only the Provided Test Environment)

In the test environment, verify whether SQL injection or weak credentials can expose sample patient information. Record only test data.

4. Recommended Secure Countermeasures

- Encrypt patient records.
- Use prepared SQL statements.
- Hash passwords.
- Enable MFA.
- Restrict access based on roles.
- Maintain audit logs and backups securely.

Conclusion

Patient information is highly sensitive and must be protected through strong confidentiality controls.

Challenge 4 – Employee Salary Application

Problem Statement

HR stores employee salary information in an application.

1. Identify Confidentiality Vulnerabilities

1. Salary information is stored without encryption.
2. Weak administrator passwords.
3. Improper authorization allows users to access salary records.

2. Explain the Attack Scenario

Step 1: An attacker guesses default credentials.

Step 2: Changing employee IDs may expose other salary records.

Step 3: Direct database access reveals salaries if data is unencrypted.

3. Demonstration (Using Only the Provided Test Environment)

In the test environment, verify whether default credentials or changing employee identifiers reveals another employee's salary information.

4. Recommended Secure Countermeasures

- Encrypt salary records.
- Enforce strong password policies.
- Implement MFA.
- Apply RBAC.
- Conduct periodic security audits.
- Monitor and log access to salary records.

Conclusion

Confidential salary information should be accessible only to authorized HR personnel.